



Workpath SDK Scan Sample Overview

Product Group: HP Workpath SDK

Version: 1.6.3

Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

HP Confidential

Table of Contents

1	Introduction	3
1.1	Document conventions	3
2	Scan Sample	4
2.1	Scan Sample on a device	5
3	Scanned file location of ScanAttributes.MeBuilder on a device	6
4	Initializing SDK.....	7
5	Integrating with scan functions.....	9
5.1	Retrieving Capabilities	9
5.1.1	Needs of Capabilities	9
5.1.2	Reading Capabilities	10
5.1.3	Getting ScannerStatus	11
5.1.4	Getting fileOptions	12
5.1.5	Getting defaults.....	13
5.2	ScanAttributes Creation	14
5.3	Submitting a job.....	15
5.3.1	Creating Job Attributes for Scan	15
5.3.2	Submitting a job.....	23
5.4	Monitoring Job.....	24
6	Legal notice	26
7	Support.....	27

1 Introduction

Scan Sample shows use case scenario how to interact with device for scanning.

1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

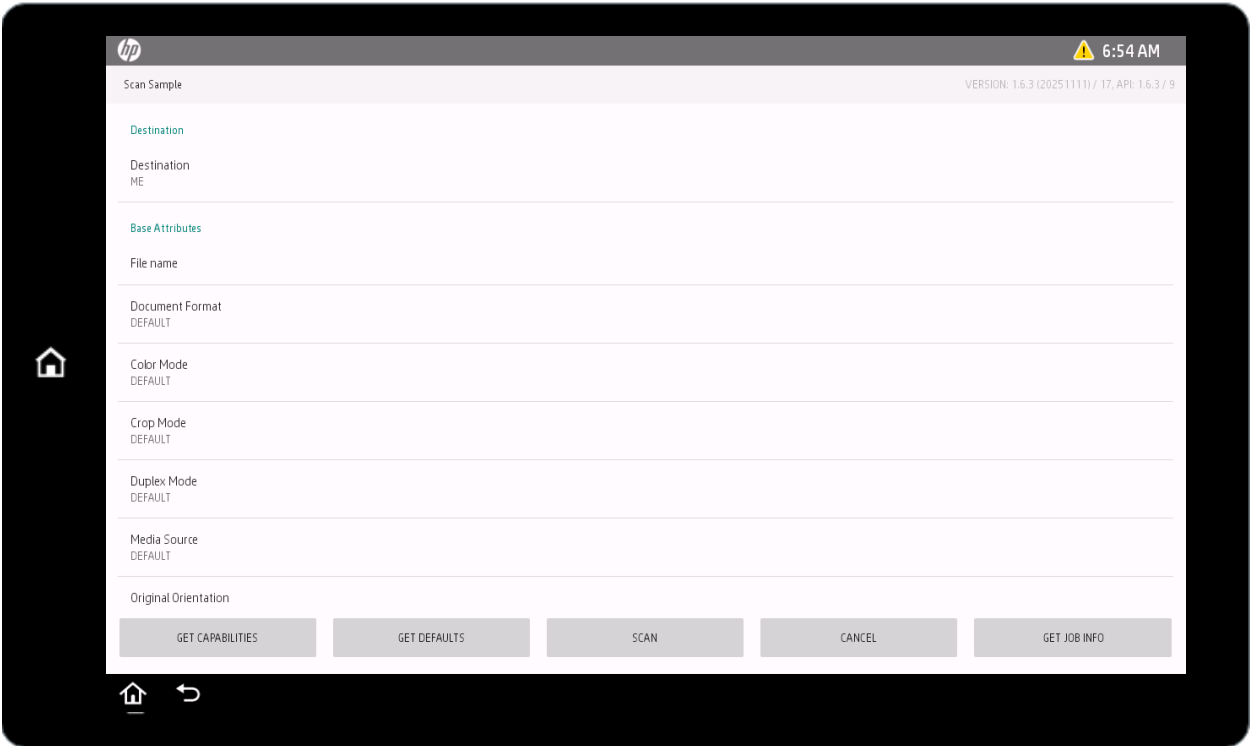
- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- `Name` - Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

2 Scan Sample

Scan Sample shows how to:

1. Interact with Scan Functions. This calls the device's scanner to:
 - Retrieve its capabilities
 - Submit scan jobs
2. Interact with Job Manager
 - Call **JobService** to monitor the current job
 - Handle **JobService.AbstractJobletObserver** callbacks

2.1 Scan Sample on a device



A main activity of Scan Sample on a device

3 Scanned file location of ScanAttributes.MeBuilder on a device

The default path of scan job using ScanAttributes.MeBuilder is "{app-private folder}/files/". setFolderName() method ignores to change the directory in different path out of application private folder.

4 Initializing SDK

Application requires initializing the SDK services when starts with an activity for accessing the SDK services. The class of `com.hp.workpath.api.Workpath` provides `initialize()` method for this.

The sample shows how to initialize SDK.

1. Call `initialize()` method of Workpath SDK on resume event, and clear it on pause event. It is for calling initialization when an activity is switched. Refer to `MainActivity.java`.

```

1. @Override
2. protected void onResume() {
3.     super.onResume();
4.
5.     // Register JobObserver to receive job state callbacks
6.     mJobObserver.register(this);
7.
8.     mContainer.setEnabled(false);
9.     mFragment = (ScanConfigureFragment) getFragmentManager().findFragmentById(R.id.data
    _container);
10.
11.     mInitializationTask = new InitializationTask(this);
12.     mInitializationTask.execute();
13. }
14.
15. @Override
16. protected void onPause() {
17.     super.onPause();
18.
19.     mInitializationTask.cancel(true);
20.     mInitializationTask = null;
21.
22.     // Unregister JobObserver
23.     mJobObserver.unregister(this);
24.
25.     if(mAlertDialog != null) {
26.         mAlertDialog.dismiss();
27.         mAlertDialog = null;
28.     }
29. }

```

Initialization call in Activity Resume

2. Make a task and call `initialize()` method with catching exceptions. If Workpath SDK is not initialized, sdk exception or security exception will be thrown. Refer to `InitializationTask.java`.

```

1. @Override
2. protected InitStatus doInBackground(final Void... params) {
3.     InitStatus status = InitStatus.NO_ERROR;
4.
5.     try {
6.         // initialize Workpath SDK
7.         Workpath.getInstance().initialize(mContextRef.get());
8.     } catch (SdkUnsupportedException sue) {
9.         Log.e(TAG, "SDK is not supported!", sue);
10.        mException = sue;
11.        status = InitStatus.INIT_EXCEPTION;
12.    } catch (SecurityException se) {
13.        Log.e(TAG, "Security exception!", se);
14.        mException = se;
15.        status = InitStatus.INIT_EXCEPTION;
16.    }
17.
18.    // Check if ScannerService is supported
19.    if (status == InitStatus.NO_ERROR
20.        && (!ScannerService.isSupported(mContextRef.get())
21.            || !JobService.isSupported(mContextRef.get()))) {
22.        // ScannerService is not supported on this device
23.        status = InitStatus.NOT_SUPPORTED;
24.    }
25.
26.    return status;
27. }

```

Exception handling by initializationTask

5 Integrating with scan functions

Scan Sample explains how to build scan attributes and submit a job. Before starting a scan, capabilities should be checked.

5.1 Retrieving Capabilities

5.1.1 Needs of Capabilities

The capability of scan service attributes is retrieved to ensure that the user is selecting one of the allowed values. Since the capability of scan attribute can vary among device's, it is very important to prevent the user from selecting unsupported attribute values as you develop your app.

5.1.2 Reading Capabilities

Reading capability requires interaction with a device and must not be invoked from the UI thread. Therefore, `MainActivity` executes `LoadCapabilitiesTask()`, which then calls the `requestCaps()` method obtaining values from `getCapabilities` of `ScannerService` to read the capabilities.

The `ScannerService.getCapabilities(context, result)` will return any errors that exist in `result` argument.

```

1.  /**
2.   * Executes request for capabilities from ScannerService.
3.   *
4.   * @param context {@link Context}
5.   * @return {@link com.hp.workpath.api.scanner.ScanAttributesCaps}
6.   */
7.  public ScanAttributesCaps requestCaps(final Context context, Result result) {
8.      if (result == null) {
9.          result = new Result();
10.     }
11.
12.     // cache capabilities for building ScanAttributes
13.     mCapabilities = ScannerService.getCapabilities(context, result);
14.
15.     if (result.getCode() == Result.RESULT_OK && mCapabilities != null) {
16.         Log.i(TAG, "Caps=" + LogUtil.getScanAttributesCapsToString(mCapabilities));
17.         Log.i(TAG, "DocumentFormatsByColorMode=" + LogUtil.getDocumentFormatsByColorMod
18. eToString(mCapabilities.getDocumentFormatsByColorMode()));
19.
20.         // get file options for defaults
21.         mFileOptionsAttributesCaps = requestFileOptionsCapabilities(
22.             ScanAttributes.ColorMode.DEFAULT, ScanAttributes.DocumentFormat.DEFAULT
23.         );
24.     } else {
25.         showResult("ScannerService.getCapabilities()", result);
26.     }
27.     return mCapabilities;
28. }

```

Loading capabilities

5.1.3 Getting ScannerStatus

The `ScannerService.getStatus(context, result)` returns current scanner status from a device.

```
1. private void getScannerStatus() {
2.     if (ScannerStatus.isSupported(getContext())) {
3.         Result result = new Result();
4.         StatusInfo statusInfo = ScannerStatus.getStatus(getContext(), result);
5.         if (result.getCode() == Result.RESULT_OK) {
6.             Log.i(TAG, "StatusInfo=" + LogUtil.getStatusInfoToString(statusInfo, true))
7.             ;
8.             mScannerStatus = LogUtil.getStatusInfoToString(statusInfo, false);
9.             mStatusTextView.setText(mScannerStatus);
10.        } else {
11.            showResult("ScannerStatus.getStatus()", result);
12.        }
13.    } else {
14.        showResult("ScannerStatus is not supported", null);
15.    }
```

Getting ScannerStatus

5.1.4 Getting fileOptions

The `ScannerService.getFileOptionsCapabilities(context, colorMode, docFormat, result)` returns capabilities based on color mode and document format.

```
1. public FileOptionsAttributesCaps requestFileOptionsCapabilities(ScanAttributes.ColorMode colorMode, ScanAttributes.DocumentFormat docFormat) {
2.     // cache file options capabilities for building FileOptionsAttributes later
3.     Result result = new Result();
4.     mFileOptionsAttributesCaps = ScannerService.getFileOptionsCapabilities(this, colorMode, docFormat, result);
5.     if (result.getCode() == Result.RESULT_OK) {
6.         Log.i(TAG, "FileOptionsAttributesCaps=" + LogUtil.getFileOptionsAttributesCapabilitiesToString(mFileOptionsAttributesCaps, colorMode, docFormat));
7.         return mFileOptionsAttributesCaps;
8.     } else {
9.         showResult("ScannerService.getFileOptionsCapabilities()", result);
10.    }
11.    return null;
12. }
```

Getting capabilities of file options

5.1.5 Getting defaults

The `ScannerService.getDefault(context, result)` returns default attributes of scan from a device.

```
1. @Override
2. protected ScanAttributes doInBackground(final Void... params) {
3.     return ScannerService.getDefault(mContextRef.get(), result);
4. }
5.
6. @Override
7. protected void onPostExecute(final ScanAttributes defaults) {
8.     super.onPostExecute(defaults);
9.
10.    if (defaults != null && result.getCode() == Result.RESULT_OK) {
11.        Log.i(TAG, "Defaults=" + LogUtil.getScanAttributesToString(defaults));
12.        mContextRef.get().showResult(mContextRef.get().getString(R.string.succeed));
13.        mFragment.get().setDefaultScanAttributes(defaults);
14.    } else {
15.        mContextRef.get().showResult("ScannerService.getDefault()", result);
16.    }
17. }
```

Getting defaults

5.2 ScanAttributes Creation

As in Scan Sample, the *PreferenceFragment* contains a variety of Sample attributes to be set manually. Please see [ScanConfigureFragment.java](#).

```

1.  /**
2.   * Simple {@link PreferenceFragmentCompat} to set Scan Attributes and save into preferences.
3.   */
4.  public final class ScanConfigureFragment extends PreferenceFragmentCompat implements
5.      SharedPreferences.OnSharedPreferenceChangeListener {
6.      private static final String TAG = MainActivity.TAG;
7.
8.      // Preferences keys for ScanAttributes
9.      public static final String PREF_DESTINATION = "pref_destination";
10.     public static final String PREF_FILE_NAME = "pref_filename";
11.     public static final String PREF_COLOR_MODE = "pref_colorMode";
12.     public static final String PREF_DUPLEX_MODE = "pref_duplexMode";
13.     public static final String PREF_RESOLUTION_TYPE = "pref_resolutionType";
14.     public static final String PREF_DOC_FORMAT = "pref_docFormat";
15.     public static final String PREF_ORG_SIZE = "pref_originalSize";
16.     public static final String PREF_CUSTOM_LENGTH = "pref_customLength";
17.     public static final String PREF_CUSTOM_WIDTH = "pref_customWidth";
18.     public static final String PREF_ORIENTATION = "pref_orientation";
19.     public static final String PREF_SCAN_PREVIEW = "pref_scanPreview";
20.     public static final String PREF_BACKGROUND_CLEANUP = "pref_backgroundCleanup";
21.     public static final String PREF_CONTRAST_ADJUSTMENT = "pref_contrastAdjustment";
22.     public static final String PREF_DARKNESS_ADJUSTMENT = "pref_darknessAdjustment";
23.     public static final String PREF_BLACK_IMAGE_REMOVAL_MODE = "pref_blankImageRemovalM
ode";
24.     public static final String PREF_COLOR_DROPOUT_MODE = "pref_colorDropoutMode";
25.     public static final String PREF_CROP_MODE = "pref_cropMode";
26.     public static final String PREF_PROGRESS_DIALOG_MODE = "pref_progressDialogMode";
27.     public static final String PREF_OUTPUT_QUALITY = "pref_outputQuality";
28.     public static final String PREF_TRANSMISSION_MODE = "pref_transmissionMode";
29.     public static final String PREF_JOB_ASSEMBLY_MODE = "pref_jobAssemblyMode";
30.     public static final String PREF_SHARPNESS_ADJUSTMENT = "pref_sharpnessAdjustment";
31.
32.     public static final String PREF_MEDIA_WEIGHT_ADJUSTMENT = "pref_mediaWeightAdjustme
nt";
32.     public static final String PREF_TEXT_PHOTO_OPTIMIZATION = "pref_textPhotoOptimizati
on";

```

Configuring scan attributes using preferenceFragment

5.3 Submitting a job

Scan job submission involves:

- Creating job attributes with Builders
 - MeBuilder
 - HttpBuilder
 - FtpBuilder
 - NetworkFolderBuilder
 - EmailBuilder
 - UsbBuilder
- Submitting the scan job
- Observing the job events and starting monitoring of job state.

5.3.1 Creating Job Attributes for Scan

ScanAttributes class provides flexibility in terms of the type of builder used to create scan attributes. The simple method for scanning a file to the `localstorage(default is {app-private folder}/files)` is called with **MeBuilder**. Other builders are also supported by **ScanAttributes** such as Ftp, Http and NetworkFolder.

Since SDK 1.1, **UsbBuilder** for scanning a file to USB has been supported.

1. MeBuilder

“MeBuilder” is for storing scanned document into local storage on a device. Sample code shows how to build attributes.

```

1.  case ME:
2.      ...
3.
4.      return new ScanAttributes.MeBuilder()
5.          .setColorMode(cm)
6.          .setDuplex(du)
7.          .setDocumentFormat(df)
8.          .setScanSize(ss)
9.          .setCustomLength(customLength)
10.         .setCustomWidth(customWidth)
11.         .setResolution(resolution)
12.         .setOrientation(orientation)
13.         .setScanPreview(scanPreview)
14.         .setBackgroundCleanup(backgroundCleanup)
15.         .setContrastAdjustment(contrastAdjustment)
16.         .setDarknessAdjustment(darknessAdjustment)
17.         .setBlankImageRemovalMode(blankImageRemovalMode)
18.         .setColorDropoutMode(colorDropoutMode)
19.         .setCropMode(cropMode)
20.         .setProgressDialogMode(progressDialogMode)
21.         .setOutputQuality(outputQuality)
22.         .setTransmissionMode(transmissionMode)
23.         .setJobAssemblyMode(jobAssemblyMode)
24.         .setSharpnessAdjustment(sharpnessAdjustment)
25.         .setMediaWeightAdjustment(mediaWeightAdjustment)
26.         .setTextPhotoOptimization(textPhotoOptimization)
27.         .setMediaSource(mediaSource)
28.         .setMisfeedDetectionMode(misfeedDetectionMode)
29.         .setFileOptionsAttributes(fileOptionsAttributes)
30.         // .setFolderName(foldername) //ignored from 1.3.1
31.         .setFileName(filename)
32.         .build(capabilities);

```

Building attributes with MeBuilder

2. HttpBuilder

“HttpBuilder” is for sending scanned document to the external http server from a device. Valid http url should be built with attributes.

```

1. case HTTP:
2.     final Uri httpUri = Uri.parse(mPrefs.getString(ScanConfigureFragment.PREF_URI_HTTP,
        ""));
3.
4.     return new ScanAttributes.HttpBuilder(httpUri)
5.         .setColorMode(cm)
6.         .setDuplex(du)
7.         .setDocumentFormat(df)
8.         .setScanSize(ss)
9.         .setCustomLength(customLength)
10.        .setCustomWidth(customWidth)
11.        .setResolution(resolution)
12.        .setOrientation(orientation)
13.        .setScanPreview(scanPreview)
14.        .setBackgroundCleanup(backgroundCleanup)
15.        .setContrastAdjustment(contrastAdjustment)
16.        .setDarknessAdjustment(darknessAdjustment)
17.        .setBlankImageRemovalMode(blankImageRemovalMode)
18.        .setColorDropoutMode(colorDropoutMode)
19.        .setCropMode(cropMode)
20.        .setProgressDialogMode(progressDialogMode)
21.        .setOutputQuality(outputQuality)
22.        .setTransmissionMode(transmissionMode)
23.        .setJobAssemblyMode(jobAssemblyMode)
24.        .setSharpnessAdjustment(sharpnessAdjustment)
25.        .setMediaWeightAdjustment(mediaWeightAdjustment)
26.        .setTextPhotoOptimization(textPhotoOptimization)
27.        .setMediaSource(mediaSource)
28.        .setMisfeedDetectionMode(misfeedDetectionMode)
29.        .setFileOptionsAttributes(fileOptionsAttributes)
30.        .setFileName(filename)
31.        .setNetworkCredentials(networkCredentialsAttributes)
32.        .build(capabilities);

```

Building attributes with HttpBuilder

3. FtpBuilder

“FtpBuilder” is for sending scanned document to the external ftp server from a device. Valid ftp server url should be built with attributes.

```

1. case FTP:
2.     final Uri ftpUri = Uri.parse(mPrefs.getString(ScanConfigureFragment.PREF_URI_FTP, "
   "));
3.
4.     return new ScanAttributes.FtpBuilder(ftpUri)
5.         .setColorMode(cm)
6.         .setDuplex(du)
7.         .setDocumentFormat(df)
8.         .setScanSize(ss)
9.         .setCustomLength(customLength)
10.        .setCustomWidth(customWidth)
11.        .setResolution(resolution)
12.        .setOrientation(orientation)
13.        .setScanPreview(scanPreview)
14.        .setBackgroundCleanup(backgroundCleanup)
15.        .setContrastAdjustment(contrastAdjustment)
16.        .setDarknessAdjustment(darknessAdjustment)
17.        .setBlankImageRemovalMode(blankImageRemovalMode)
18.        .setColorDropoutMode(colorDropoutMode)
19.        .setCropMode(cropMode)
20.        .setProgressDialogMode(progressDialogMode)
21.        .setOutputQuality(outputQuality)
22.        .setTransmissionMode(transmissionMode)
23.        .setJobAssemblyMode(jobAssemblyMode)
24.        .setSharpnessAdjustment(sharpnessAdjustment)
25.        .setMediaWeightAdjustment(mediaWeightAdjustment)
26.        .setTextPhotoOptimization(textPhotoOptimization)
27.        .setMediaSource(mediaSource)
28.        .setMisfeedDetectionMode(misfeedDetectionMode)
29.        .setFileOptionsAttributes(fileOptionsAttributes)
30.        .setFileName(filename)
31.        .setNetworkCredentials(networkCredentialsAttributes)
32.        .build(capabilities);

```

Building attributes with FtpBuilder

4. NetworkFolderBuilder

“NetworkFolderBuilder” is for sending scanned document to the external network folder location from a device. Valid networkFolder url should be built with attributes.

```

1. case NETWORK_FOLDER:
2.     final Uri networkFolderUri = Uri.parse(mPrefs.getString(ScanConfigureFragment.PREF_
   URI_NETWORK_FOLDER, ""));
3.
4.     return new ScanAttributes.NetworkFolderBuilder(networkFolderUri)
5.         .setColorMode(cm)
6.         .setDuplex(du)
7.         .setDocumentFormat(df)
8.         .setScanSize(ss)
9.         .setCustomLength(customLength)
10.        .setCustomWidth(customWidth)
11.        .setResolution(resolution)
12.        .setOrientation(orientation)
13.        .setScanPreview(scanPreview)
14.        .setBackgroundCleanup(backgroundCleanup)
15.        .setContrastAdjustment(contrastAdjustment)
16.        .setDarknessAdjustment(darknessAdjustment)
17.        .setBlankImageRemovalMode(blankImageRemovalMode)
18.        .setColorDropoutMode(colorDropoutMode)
19.        .setCropMode(cropMode)
20.        .setProgressDialogMode(progressDialogMode)
21.        .setOutputQuality(outputQuality)
22.        .setTransmissionMode(transmissionMode)
23.        .setJobAssemblyMode(jobAssemblyMode)
24.        .setSharpnessAdjustment(sharpnessAdjustment)
25.        .setMediaWeightAdjustment(mediaWeightAdjustment)
26.        .setTextPhotoOptimization(textPhotoOptimization)
27.        .setMediaSource(mediaSource)
28.        .setMisfeedDetectionMode(misfeedDetectionMode)
29.        .setFileOptionsAttributes(fileOptionsAttributes)
30.        .setFileName(filename)
31.        .setNetworkCredentials(networkCredentialsAttributes)
32.        .build(capabilities);

```

Building attributes with NetworkFolderBuilder

5. EmailBuilder

“EmailBuilder” is for sending email with scanned document as attachment from a device. If smtpconfiguration is not provided, device will send an email based on a device configuration that can be checked through EWS.

```

1.  case EMAIL:
2.      String emailTo = mPrefs.getString(ScanConfigureFragment.PREF_EMAIL_TO, "");
3.      String emailCc = mPrefs.getString(ScanConfigureFragment.PREF_EMAIL_CC, "");
4.      String emailBcc = mPrefs.getString(ScanConfigureFragment.PREF_EMAIL_BCC, "");
5.      String emailFrom = mPrefs.getString(ScanConfigureFragment.PREF_EMAIL_FROM, "");
6.      String emailSubject = mPrefs.getString(ScanConfigureFragment.PREF_EMAIL_SUBJECT, ""
7.  );
8.      String emailMessage = mPrefs.getString(ScanConfigureFragment.PREF_EMAIL_MESSAGE, ""
9.  );
10.     boolean isEnabledSmtP = mPrefs.getBoolean(ScanConfigureFragment.PREF_EMAIL_SMTP, fa
11. lse);
12.
13.     EmailAttributes.Builder emailAttributesBuilder = new EmailAttributes.Builder();
14.
15.     if (!TextUtils.isEmpty(emailTo)) {
16.         emailAttributesBuilder.addToAddresses(emailTo.split(","));
17.     }
18.     ...
19.
20.     SmtPAttributes smtPAttributes = null;
21.     if (isEnabledSmtP) {
22.         ...
23.
24.         NetworkCredentialsAttributes authentication = null;
25.         if (!TextUtils.isEmpty(username)) {
26.             authentication = new NetworkCredentialsAttributes.Builder()
27.                 .setUserName(username)
28.                 .setPassword(password)
29.                 .build();
30.         }
31.
32.         SmtPAttributes.TransportMode transportMode = SmtPAttributes.TransportMode.value
33. Of(transportModeStr);
34.
35.         smtPAttributes = new SmtPAttributes.Builder(hostname)
36.             .setPort(Integer.parseInt(port))
37.             .setConnectTimeout(Integer.parseInt(connectionTimeout))
38.             .setReadTimeout(Integer.parseInt(readTimeout))
39.             .setServerCredentials(authentication)
40.             .setTransportMode(transportMode)
41.             .build();
42.     }
43.
44.     return new ScanAttributes.EmailBuilder(emailAttributesBuilder.build())
45.         .setSmtPAttributes(smtPAttributes)
46.         .setColorMode(cm)
47.         .setDuplex(du)
48.         .setDocumentFormat(df)
49.         .setScanSize(ss)
50.         .setCustomLength(customLength)
51.         .setCustomWidth(customWidth)
52.         .setResolution(resolution)
53.         .setOrientation(orientation)

```

```
50.         .setScanPreview(scanPreview)
51.         .setBackgroundCleanup(backgroundCleanup)
52.         .setContrastAdjustment(contrastAdjustment)
53.         .setDarknessAdjustment(darknessAdjustment)
54.         .setBlankImageRemovalMode(blankImageRemovalMode)
55.         .setColorDropoutMode(colorDropoutMode)
56.         .setCropMode(cropMode)
57.         .setProgressDialogMode(progressDialogMode)
58.         .setOutputQuality(outputQuality)
59.         .setTransmissionMode(transmissionMode)
60.         .setJobAssemblyMode(jobAssemblyMode)
61.         .setSharpnessAdjustment(sharpnessAdjustment)
62.         .setMediaWeightAdjustment(mediaWeightAdjustment)
63.         .setTextPhotoOptimization(textPhotoOptimization)
64.         .setMediaSource(mediaSource)
65.         .setMisfeedDetectionMode(misfeedDetectionMode)
66.         .setFileOptionsAttributes(fileOptionsAttributes)
67.         .setFileName(filename)
68.         .build(capabilities);
```

Building attributes with EmailBuilder

6. UsbBuilder

“**UsbBuilder**” is for sending a scanned document into USB. Valid file location of USB that is retrieved by **massstorage** should be built with attributes.

```
1. case USB:
2.     final String usbLocation = mPrefs.getString(ScanConfigureFragment.PREF_USB_LOCATION
3.         , "");
4.     return new ScanAttributes.UsbBuilder(usbLocation)
5.         .setColorMode(cm)
6.         .setDuplex(du)
7.         .setDocumentFormat(df)
8.         .setScanSize(ss)
9.         .setResolution(resolution)
10.        .setOrientation(orientation)
11.        .setScanPreview(scanPreview)
12.        .setBackgroundCleanup(backgroundCleanup)
13.        .setContrastAdjustment(contrastAdjustment)
14.        .setDarknessAdjustment(darknessAdjustment)
15.        .setBlankImageRemovalMode(blankImageRemovalMode)
16.        .setColorDropoutMode(colorDropoutMode)
17.        .setCropMode(cropMode)
18.        .setProgressDialogMode(progressDialogMode)
19.        .setOutputQuality(outputQuality)
20.        .setTransmissionMode(transmissionMode)
21.        .setJobAssemblyMode(jobAssemblyMode)
22.        .setFileOptionsAttributes(fileOptionsAttributes)
23.        .setFileName(filename)
24.        .build(capabilities);
```

Building attributes with UsbBuilder

5.3.2 Submitting a job

Sample shows how an application creates scan job and checks job status. Main method for submitting a job is `ScannerService.submit()`. This method allows to put `ScanletAttributes` as a parameter to show embedded settings dialog.

The following code shows how to implement it with `ScanletAttributes`.

```
1. final ScanletAttributes taskAttribs = new ScanletAttributes.Builder()
2.     .setShowSettingsUi(settingsUi)
3.     .setAllowMultipleScan(allowMultipleScan)
4.     .build();
5.
6. // Submit the job
7. final String rid = ScannerService.submit(activity, attributes, taskAttribs);
8.
9. Log.i(TAG, "Job submitted with rid = " + rid);
```

Submitting a scan job with attributes

5.4 Monitoring Job

Callbacks for Scan job are received via `JobService.AbstractJobletObserver`. Scan Sample extends this class to obtain callbacks and the state of job. Registration and unregistration of the observer are done by `OnResume()` / `OnPause()` respectively.

`MainActivity` contains `JobObserver` implementation:

```

1.  /**
2.   * Observer for submitted job
3.   */
4.   private class JobObserver extends JobService.AbstractJobletObserver {
5.       public JobObserver(final Handler handler) {
6.           super(handler);
7.       }
8.
9.       @Override
10.      public void onProgress(final String rid, final JobInfo jobInfo) {
11.          Log.i(TAG, "onProgress: Received rid=" + rid);
12.          Log.i(TAG, "JobInfo=" + LogUtil.getJobInfoToString(jobInfo));
13.          if (rid.equals(mRid)) {
14.              if (mJobId == null) {
15.                  if (jobInfo.getJobId() != null) {
16.                      mJobId = jobInfo.getJobId();
17.
18.                      Log.i(TAG, "Received jobId=" + mJobId);
19.                      showSnackBar(getString(R.string.job_id, mJobId));
20.
21.                      final SharedPreferences prefs = PreferenceManager.getDefaultShare
redPreferences(getApplicationContext());
22.                      final boolean monitorJob = prefs.getBoolean(ScanConfigureFragme
nt.PREF_MONITOR_JOB, true);
23.
24.                      if (monitorJob) {
25.                          prefs.edit().putString(ScanConfigureFragment.CURRENT_JOB_ID
, mJobId).apply();
26.
27.                          final boolean showProgress =
28.                              prefs.getBoolean(ScanConfigureFragment.PREF_SHOW_JO
B_PROGRESS, true);
29.
30.                          // Monitor the job completion
31.                          final JobletAttributes taskAttributes =
32.                              new JobletAttributes.Builder().setShowUi(showProgre
ss).build();
33.
34.                          final Intent intent = new Intent(getApplicationContext(), J
obCompleteReceiver.class);
35.                          intent.setAction(ACTION_SCAN_COMPLETED);
36.                          intent.putExtra(JobCompleteReceiver.RID_EXTRA, rid);
37.                          intent.putExtra(JobCompleteReceiver.JOB_ID_EXTRA, mJobId);
38.
39.                          final String jrid = JobService.monitorJobInForeground(MainA
ctivity.this, mJobId,
40.                              taskAttributes, intent);
41.
42.                          Log.i(TAG, "MonitorJob request: " + jrid);
43.                      }

```



```

44.         }
45.     }
46. }
47.
48.
49. @Override
50. public void onComplete(final String rid, final JobInfo jobInfo) {
51.     Log.i(TAG, "onComplete: Received rid=" + rid);
52.     Log.i(TAG, "JobInfo=" + LogUtil.getJobInfoToString(jobInfo));
53.     if (jobInfo.getJobType() == JobInfo.JobType.SCAN) {
54.         ScanJobData scanJobData = jobInfo.getJobData();
55.         final List<String> images = scanJobData.getFileNames();
56.
57.         if (images != null && images.size() > 0) {
58.             showSnackBar("Scan completed! Image path is " + Arrays.toString(ima
ges.toArray()));
59.         } else {
60.             showSnackBar(getString(R.string.job_completed));
61.         }
62.     }
63. }
64.
65. @Override
66. public void onFail(final String rid, final Result result) {
67.     Log.e(TAG, "onFail: Received rid=" + rid);
68.     showResult(getString(R.string.job_failed), result);
69. }
70.
71. @Override
72. public void onCancel(final String rid) {
73.     Log.i(TAG, "onCancel: Received rid=" + rid);
74.     showSnackBar(getString(R.string.job_cancelled));
75. }
76. }

```

Receiving events of a scan job state

NOTE: `JobService.monitorJobInForeground()` provides a progress dialog for showing job state by Workpath SDK. If application doesn't have custom dialog, this api will help to provide feature easily. Sample also explains how to call the method.

6 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2018 HP Development Company, L.P.

7 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.